Faust : composer des programmes

Atelier — IEEE IS2 2026

Yann Orlarey

Cannes, 30 octobre 2026

EMERAUDE (INRIA/INSA/GRAME)

Bienvenue

Avant de commencer

- Un navigateur suffit : ordinateur **ou** smartphone
- Chaque exemple a son QR code : scannez, jouez, modifiez
- Pas d'écouteurs : nous jouerons **ensemble**
- iPhone : jouez avec les QR codes Wahoo et Kisana (applications web)

Au programme :

1. Faust en 5 minutes, et d'où il vient
2. Composer des programmes
3. Personnaliser sans modifier
4. La *widget modulation*
5. Démo : une IA et une corne de brume
6. Conclusion : le rôle de la notation



orlarey.github.io/
faustcode

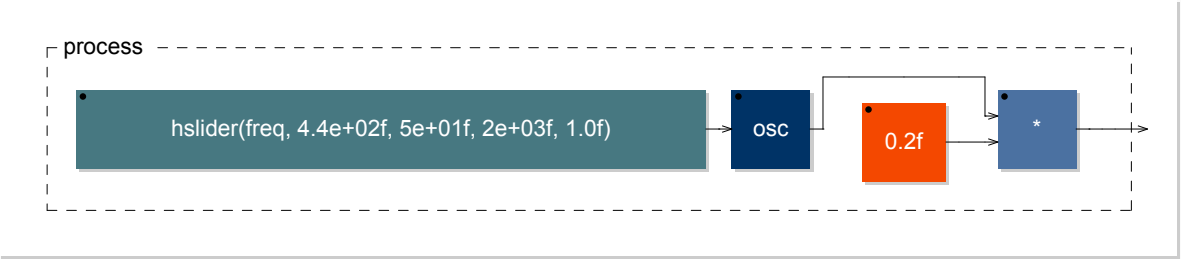
Faust en 5 minutes

Qu'est-ce que Faust ?

- Un **langage fonctionnel** dédié au traitement du signal audio temps réel
- Conçu au GRAME (Lyon) au début des années 2000 ; libre (GPL / LGPL)
- Un programme Faust décrit un **circuit** : une fonction de signaux vers des signaux
- Le compilateur produit du code optimisé : C++, C, Rust, WebAssembly, LLVM, VHDL, Cmajor...
- ... et des applications complètes grâce aux *architectures* : plugins, PWA, Android, iOS, Bela, Daisy, ESP32, Teensy, OWL, FPGA...

Un premier programme

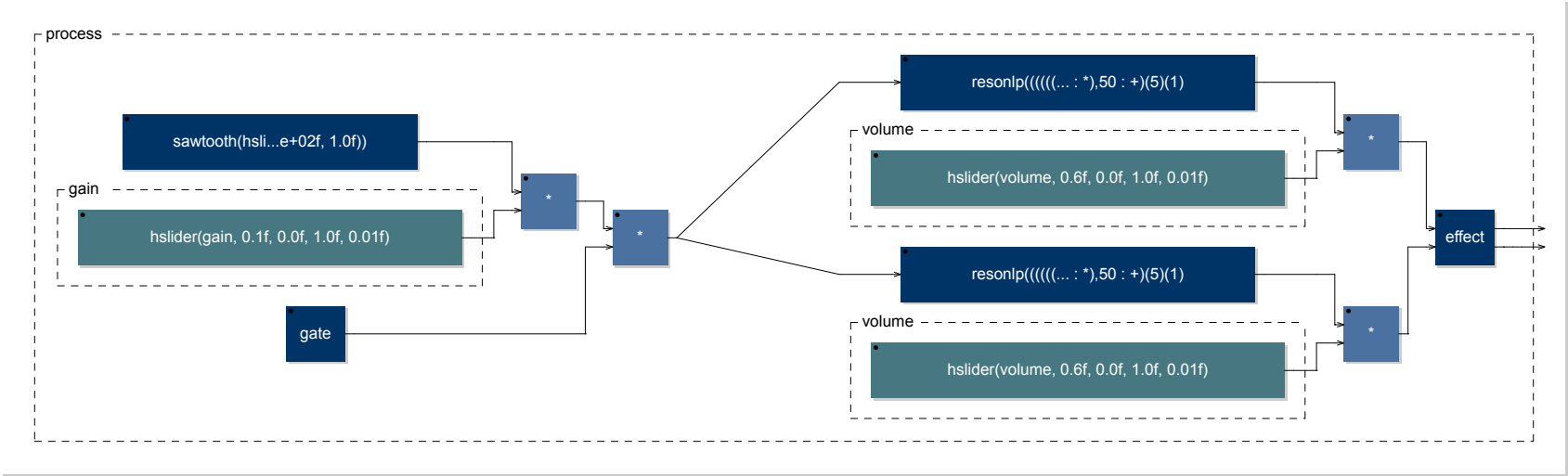
```
import("stdfaust.lib");
process = hslider("freq", 440, 50, 2000, 1) : os.osc : *(0.2);
```



Ouvrir dans faustcode

- `process` est le point d'entrée, comme `main` en C
- Le programme se lit comme le circuit : slider : oscillateur : gain
- L'interface graphique est **déduite** du programme

À vous : Wahoo sur votre smartphone



- Scie, deux filtres résonants modulés par des LFO, écho, limiteur : 20 lignes de Faust
- Inclinez le téléphone : `[acc: ...]` relie deux sliders aux **accéléromètres**
- Même source : navigateur, PWA, application Android

Ouvrir dans faustcode

D'où vient Faust ?

Logique combinatoire



- M. Schönfinkel, *Über die Bausteine der mathematischen Logik*, 1924 ; puis H. B. Curry
- Idée : se passer complètement des **variables**
- Trois combinateurs suffisent :

$$S\,x\,y\,z \to x\,z\,(y\,z) \qquad K\,x\,y \to x \qquad I\,x \to x$$

- En Faust : `_` (identité), `!` (oubli), `:` , `<:` … on assemble des circuits **sans nommer les signaux**

Lambda-calcul

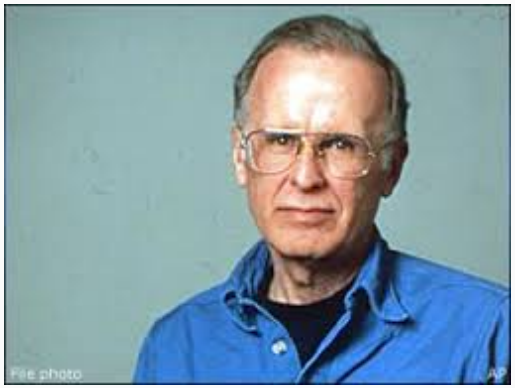


- A. Church, *The Calculi of Lambda-Conversion*, 1941
- Un langage réduit à l'essentiel : **abstraction** et **application**
- Les fonctions sont des valeurs comme les autres

```
square = \(x).(x*x);        // abstraction
process = square(0.5);      // application
```

- Faust est un langage fonctionnel : les circuits sont des valeurs, que l'on passe, renvoie et assemble

FP et l'algèbre de programmes



- J. Backus, *Can Programming Be Liberated from the von Neumann Style?*, Turing Award 1977
- Programmer en **composant** des fonctions, avec des lois algébriques

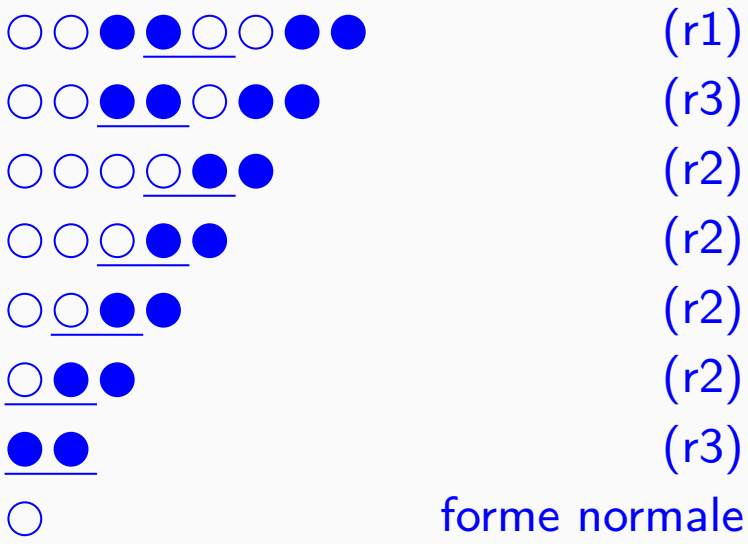
FP	`[+,*]:<2,3>`
Faust	`2,3 <: +,*`

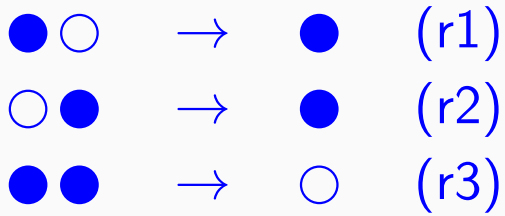
Systèmes de réécriture (Axel Thue, 1914)

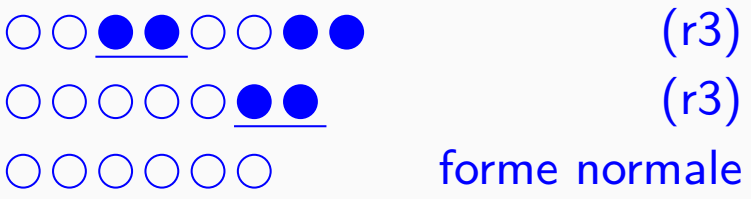
Décrire des transformations par de simples **règles de remplacement**.

Une rangée de grains de café, et trois règles :









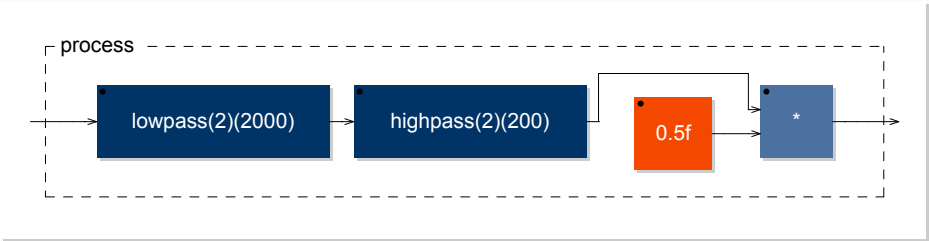
Deux ordres d'application, deux formes normales : ce système n'est pas **confluent**. Un système confluent définit un calcul.

Réécriture en Faust

- Calcul par **règles** : on remplace une expression qui *filtre* un motif par une autre
- En Faust, des règles évaluées **à la compilation**, qui filtrent la structure des circuits :

```
serial((x,xs)) = x : serial(xs);
serial(x)      = x;

process = serial((fi.lowpass(2,2000),
                  fi.highpass(2,200), *(0.5)));
```

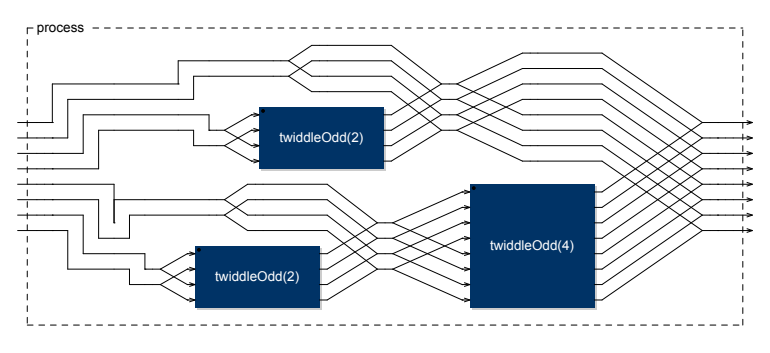


- Méta-langage Turing-complet : il fabrique des circuits, puis disparaît

Ouvrir dans faustcode

Réécriture : la FFT de Julius Smith

```
fftb(1) = _,_;                          // un nombre complexe (re, im)
fftb(N) = si.cbus(N)
        : (fftb(No2) <: (si.cbus(No2), si.cbus(No2))),
          (fftb(No2) <: (si.cbus(N) : twiddleOdd(N)))
        :> si.cbus(N)
with { No2 = int(N)>>1; ... };
```



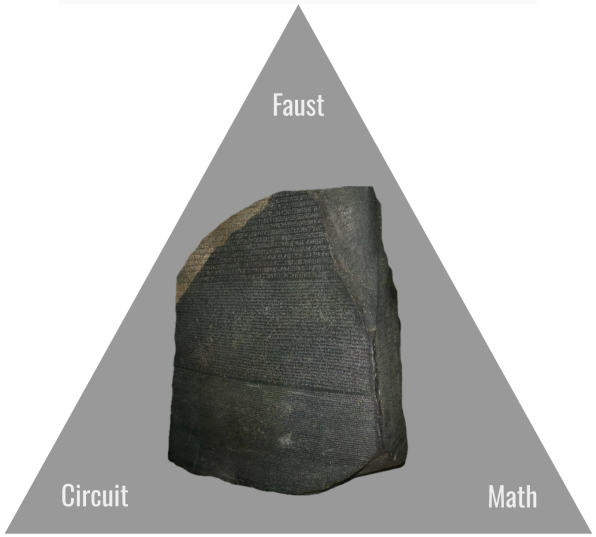
- FFT radix 2 de Cooley-Tukey en deux règles (`analyzers.lib`) ; le compilateur déplie la récursion : ici `fftb(4)`

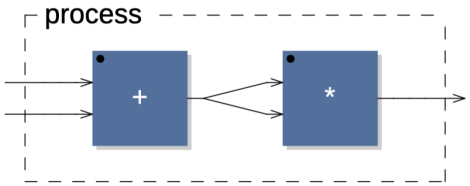
Ouvrir dans faustcode

Un programme est un circuit

Trois lectures d'un même programme

```
process = + <: *;
```

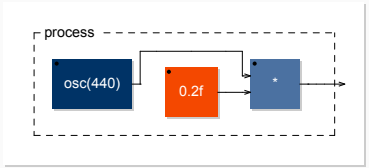
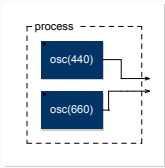
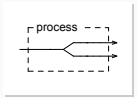
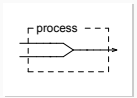
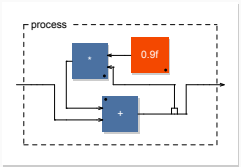




$$y(t) = (x_1(t) + x_2(t))^2$$

- Le code, le diagramme et la formule : les deux derniers sont produits par le compilateur
- Signal = fonction du temps ; circuit = fonction de signaux ; opérateur = fonction de circuits

Cinq opérateurs pour assembler des circuits

Opérateur	Nom	Exemple	Circuit
`A : B`	séquence	`os.osc(440) : *(0.2)`	
`A , B`	parallèle	`os.osc(440), os.osc(660)`	
`A <: B`	éclatement	`_ <: _, _`	
`A :> B`	regroupement	`_, _ :> _`	
`A ~ B`	récursion	`+ ~ *(0.9)`	

Composer des programmes

Le pipe Unix

> *We should have some ways of coupling programs like garden hose — screw in another segment when it becomes necessary to massage data in another way.*
> *M. D. McIlroy, 11 octobre 1964*

```
ls | grep dsp | wc -l
```

- Des programmes indépendants, assemblés grâce à une interface uniforme : un flux de texte
- `f : g` en Faust `f | g` en Unix $g \circ f$ en mathématiques

Et en C++ ?

- Deux programmes C++ = deux fonctions `main()`
- Pour les combiner dans un troisième, il faut les **réécrire** :
  - les transformer en bibliothèques,
  - définir une interface commune (tampons, formats, horloge),
  - écrire le code d'assemblage et l'interface graphique
- Un programme C++ n'est pas une valeur du langage

Deux programmes indépendants

`synth.dsp` : une scie dont on règle la fréquence

```
import("stdfaust.lib");
freq = hslider("freq", 220, 50, 1000, 1);
process = os.sawtooth(freq) * 0.2;
```

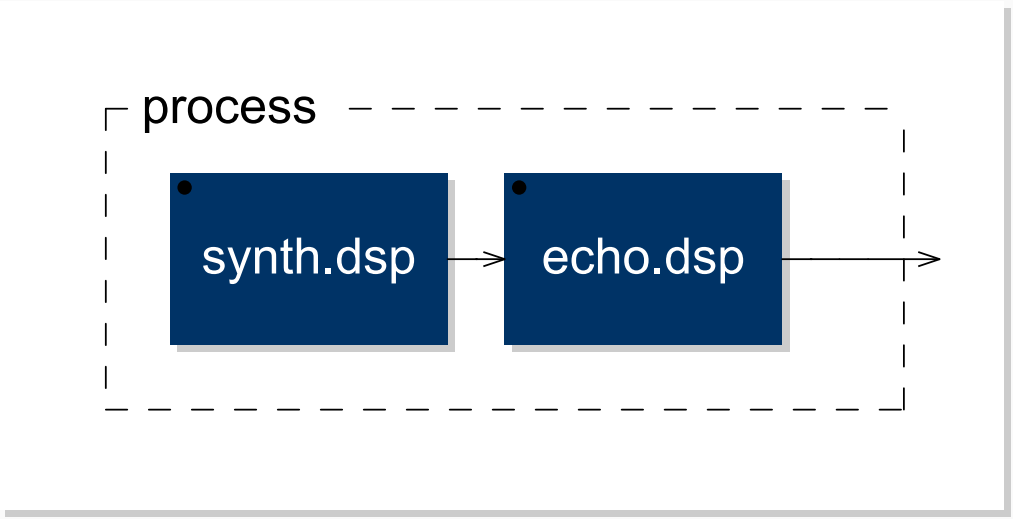
`echo.dsp` : un écho avec rétroaction

```
import("stdfaust.lib");
d  = hslider("delay", 0.25, 0, 1, 0.01);
fb = hslider("feedback", 0.5, 0, 0.95, 0.01);
process = + ~ (de.delay(ma.SR, d*ma.SR) * fb);
```

Un troisième programme : `component`

`combo.dsp`

```
process = component("synth.dsp") : component("echo.dsp");
```



- Chaque programme devient un circuit, assemblé par l'opérateur :
- Les interfaces sont **fusionnées** automatiquement : `freq`, `delay`, `feedback`

Ouvrir dans faustcode

Tous les opérateurs s'appliquent

```
// en parallèle : deux synthés indépendants, sortie stéréo
process = hgroup("L", component("synth.dsp")),
          hgroup("R", component("synth.dsp"));

// une source, deux traitements
process = component("synth.dsp") <: component("echo.dsp"), _;

// rétroaction entre programmes, à l'échantillon près
process = component("synth.dsp")
        : + ~ (component("echo.dsp") : *(0.3));
```

- Deux widgets de même chemin sont **partagés** : sans les `hgroup`, un seul slider `freq`
- Composition **à la compilation** : le résultat est un seul programme optimisé
- Une boucle `~` coûte **un échantillon** de retard ; entre plugins d'un hôte audio, c'est au moins un bloc

À vous : composer

Dans un seul fichier, chaque programme devient un `environment` :

```
synth = environment { ... };
echo  = environment { ... };

process = synth.process
        : echo.process;
```

`component("f.dsp")` équivaut à `library("f.dsp").process`



Ouvrir dans faustcode

Personnaliser sans modifier

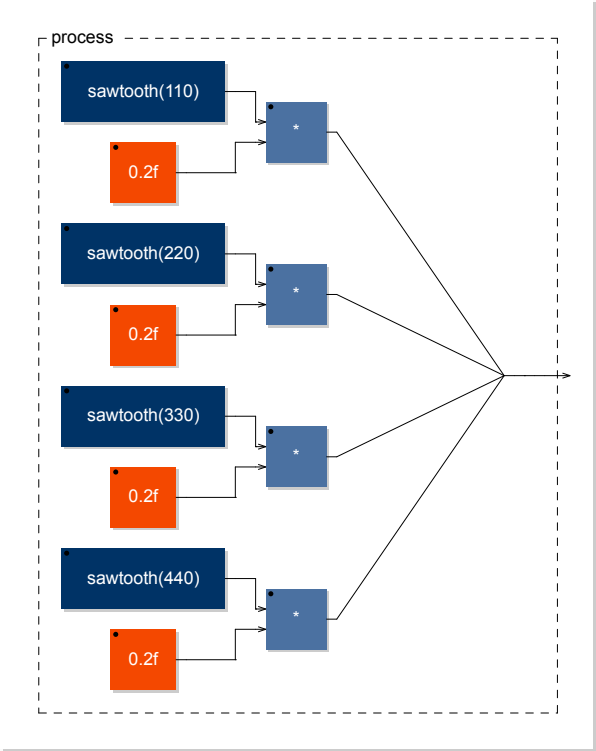
La substitution explicite

```
process = component("synth.dsp")[freq = 110 + 50*os.osc(0.5);]
        : component("echo.dsp");
```

- `[ ... ]` **redéfinit** une définition interne du programme, sans toucher à son fichier
- Ici, le slider `freq` est remplacé par un LFO : il **disparaît** de l'interface
- Possible parce que chaque expression garde son **environnement lexical** (fermeture)

Substitution et répétition

```
process = par(i, 4, component("synth.dsp")[freq = 110*(i+1);]) :
```



- Quatre copies du même programme, chacune réglée sur un harmonique
- Une synthèse additive en une ligne
- Le compilateur résout tout : il ne reste que les quatre oscillateurs

Ouvrir dans faustcode

À vous : substituer



Substitution par un LFO

Ouvrir dans faustcode



Synthèse additive

Ouvrir dans faustcode

La *widget modulation*

Le problème

- Je veux moduler la fréquence de `synth.dsp` avec un LFO
- Le slider `freq` est **à l'intérieur** du programme : aucune entrée ne permet d'agir dessus
- La substitution le remplace, mais je perds le contrôle manuel
- Sans mécanisme dédié, il faut **modifier le code** du programme

Moduler un widget de l'extérieur

```
["freq" -> component("synth.dsp")]
```

- Ajoute une **entrée** au circuit, qui module le slider `freq`
- Par défaut, le signal de modulation **multiplie** la valeur du slider
- Le code de `synth.dsp` n'est pas modifié

Forme	Effet
`["freq" -> C]`	nouvelle entrée, combinée par `*`
`["freq":+ -> C]`	nouvelle entrée, combinée par `+`
`["freq":*(lfo) -> C]`	modulation interne, sans nouvelle entrée
`["freq":440 -> C]`	le slider est remplacé par une constante

À vous : moduler

```
lfo = 1 + 0.5*os.osc(
  hslider("lfo", 3, 0.1, 20, 0.1));

process = ["freq":*(lfo)
              -> synth.process]
        : echo.process;
```

Ouvrir dans faustcode

Démo : une IA, un DSL et une corne de brume

Le défi

- Reproduire en Faust le son d'une **corne de brume** à partir d'un enregistrement (22 s)
- Écoutons la cible
- Difficulté : un modèle de langage **n'entend pas**
  - il peut raisonner sur un son, écrire du DSP, mesurer un spectre…
  - … mais le jugement « est-ce que ça sonne juste ? » lui échappe
- Question : quelle **rétroaction** compense au mieux cette surdité ?

Le dispositif

Acteur	Rôle
Claude Code	lit et écrit les fichiers, analyse en Python la cible et ses propres rendus
faustcode (navigateur + pont MCP)	compile et joue chaque version ; `render_audio` : rendu hors ligne, métriques par phase, WAV
Faust	la notation : un modèle complet en 60 lignes, vérifié par le compilateur
Moi	l'oreille, quand l'IA la demande

La consigne

Model this foghorn in Faust. Do not fall into the trap of static additive synthesis — faithfully reproduce the full spectral evolution, especially the attack. Work by measurement: first extract the target's characteristics (envelope by phase, f with sub-harmonic correction, harmonic profile, HNR, roughness), then iterate by comparing your rendering to the target on these same metrics at every step. Decide on the synthesis approach based on what the measurements say, and let me know when you need my ear.

Démo

En direct : Claude Code + faustcode

Ce que l'IFC 2026 nous avait appris

- **Le piège du f** : le 2e harmonique domine ; « le pic le plus fort » donne 262 Hz au lieu de 131 Hz
- **Mesurer par phase** (attaque / tenue / extinction) : la « meilleure » version avait une extinction de 48 ms contre 907 ms pour la cible, défaut invisible sur le spectre de la tenue
- **Les métriques rejoignent l'oreille** : le classement chiffré des versions reproduisait mon jugement d'écoute
- **Additif affiné ou modèle physique ?** C'est l'additif affiné qui l'a emporté : « différent, mais presque mieux »

Version finale de l'IFC 2026 : **Ouvrir dans faustcode**

Conclusion

Pourquoi Faust permet cela

- **Sémantique fonctionnelle** : un programme est un circuit, et les circuits forment une algèbre (5 opérateurs)
- **Environnements lexicaux** : `component`, `library`, `environment` et la substitution `[...]` manipulent des programmes comme des valeurs
- **Composition à la compilation** : pas de coût à l'exécution, rétroaction à l'échantillon près
- **Interfaces déduites** du code : elles se composent avec lui, et la *widget modulation* les rend modulables

La notation, outil de pensée

- Une notation n'est pas qu'un moyen de communiquer ou de se souvenir : c'est un **outil cognitif**
- L'algèbre a rendu manipulables des objets impensables en langue naturelle (équations différentielles, transformées)
- La notation musicale a rendu possibles les grandes formes, comme la symphonie
- **Faust est une notation pour les algorithmes de traitement du signal**, directement reliée à leur implémentation efficace

Pourquoi un DSL à sémantique forte aide aussi les IA

	Faust	C++
Rapport sens / bruit syntaxique	le code décrit l'algorithme	tampons, boucles, structures
Composition	`A : B`	gestion explicite des tampons et des blocs
État	pas d'état mutable explicite	état mutable partout
Taille	quelques dizaines de lignes	plusieurs fichiers, des centaines de lignes

- Le programme entier tient dans la **fenêtre de contexte**
- Le compilateur **vérifie** et fournit des vues (diagrammes, signaux, erreurs) : autant de retours structurés

En résumé

Comme nous, les IA ont besoin de **notations** et de **boucles de rétroaction** pour créer.

- Une notation compacte, à la sémantique forte : Faust
- Un protocole pour agir et observer : MCP
- Des mesures pour remplacer l'oreille... et l'oreille humaine pour trancher

Des prompts aux plugins : l'industrie passe par Faust

	**Pluginmaker.ai**	plugins VST/AU créés à partir d'une description en langage naturel
	**Stratus** (Chaos Audio)	*AI FX Builder* : des effets décrits par un prompt (depuis mars 2026)
	**MAGDA**	effets écrits en Faust, panneau IA qui génère du Faust
	**Incant Audio**	prompt → LLM → code Faust → effet VST3 temps réel
	**Persistent Faust**	assistant GPT intégré pour écrire et améliorer du Faust

Source : https://faust.grame.fr/community/powered-by-faust/

Pour aller plus loin

- Site : https://faust.grame.fr
- Éditeur en ligne : https://faustide.grame.fr
- Documentation : https://faustdoc.grame.fr
- Bibliothèques : https://faustlibraries.grame.fr
- faustcode : https://orlarey.github.io/faustcode/
- Atelier *widget modulation* : https://effractionmusicale.fr/modulation

Merci !



Kisana, à jouer sur votre téléphone